NLP LAB PROJECT · RETRIEVAL-AUGMENTED GENERATION

College Knowledge Assistant

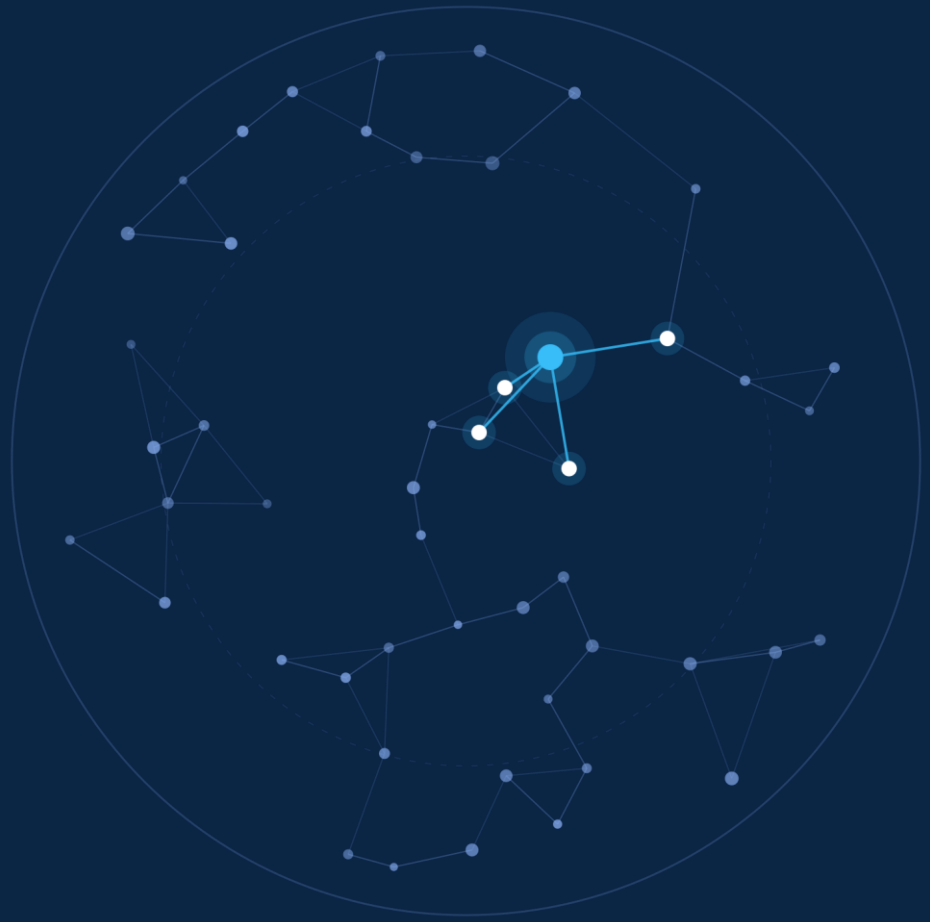
A RAG application that answers questions from your PDFs, running 100% locally

Python | LangChain | FAISS | Ollama | Streamlit

Team: [Member 1] · [Member 2] · [Member 3] · [Member 4]

Guide: [Faculty Name]

Birla Institute of Technology, Mesra



Why a College Knowledge Assistant?

The problem

Information is buried in PDFs

Notes, syllabi and notices mean page after page of scrolling.

Keyword search misses meaning

Ctrl+F finds exact words, not the answer to a question.

General LLMs don't know our college

College-specific facts aren't in their training data, so they may guess.

OUR SOLUTION

Ask in plain English. Get answers grounded in your PDF.

- Answers only from the uploaded PDF
- Shows source pages + retrieved text
- Says “not available” if the PDF lacks it
- Runs offline on a laptop with Ollama

EXAMPLE FROM OUR TEST RUN

“Who proposed Word2Vec and when?”

“According to the document, Word2Vec was proposed by Mikolov et al. in 2013.”

Sources shown: pages 10–13 of the NLP lecture PDF

What is Retrieval-Augmented Generation?

Give the LLM the relevant pages first, then let it answer: an open-book exam instead of answering from memory.

1

Retrieve

Find the document chunks most similar to the question

2

Augment

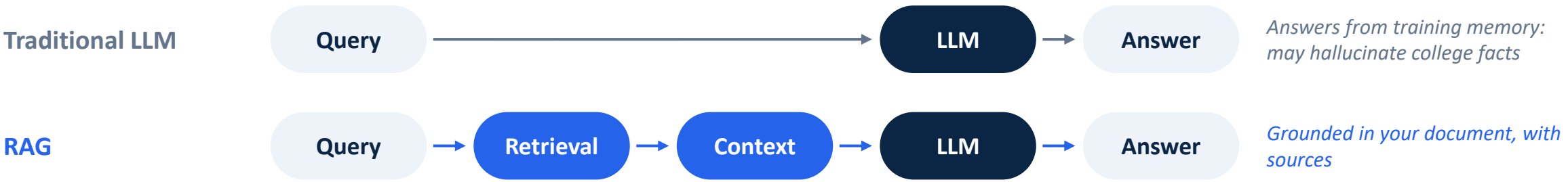
Insert those chunks into the prompt as CONTEXT

3

Generate

The LLM writes the answer using only that context

Traditional LLM vs RAG



Technology Stack (as implemented)

Python 3.14

`rag_pipeline.py · app.py`

Whole project: RAG logic in one file, UI in another

pypdf

`PdfReader`

Extracts the text of each PDF page; blank pages skipped

LangChain

`core · text-splitters · ollama`

Document objects, prompt template and the LCEL chain

Text Splitter

`RecursiveCharacterTextSplitter`

Splits pages into 500-char chunks with 100-char overlap

nomic-embed-text

`OllamaEmbeddings · 768-d`

Embedding model: turns chunks and questions into vectors

FAISS

`faiss-cpu · IndexFlatIP`

Stores the vectors; exact cosine search finds the 4 best chunks

Llama 3.2 (3B)

`ChatOllama · temperature 0`

Local LLM that writes the answer from retrieved context

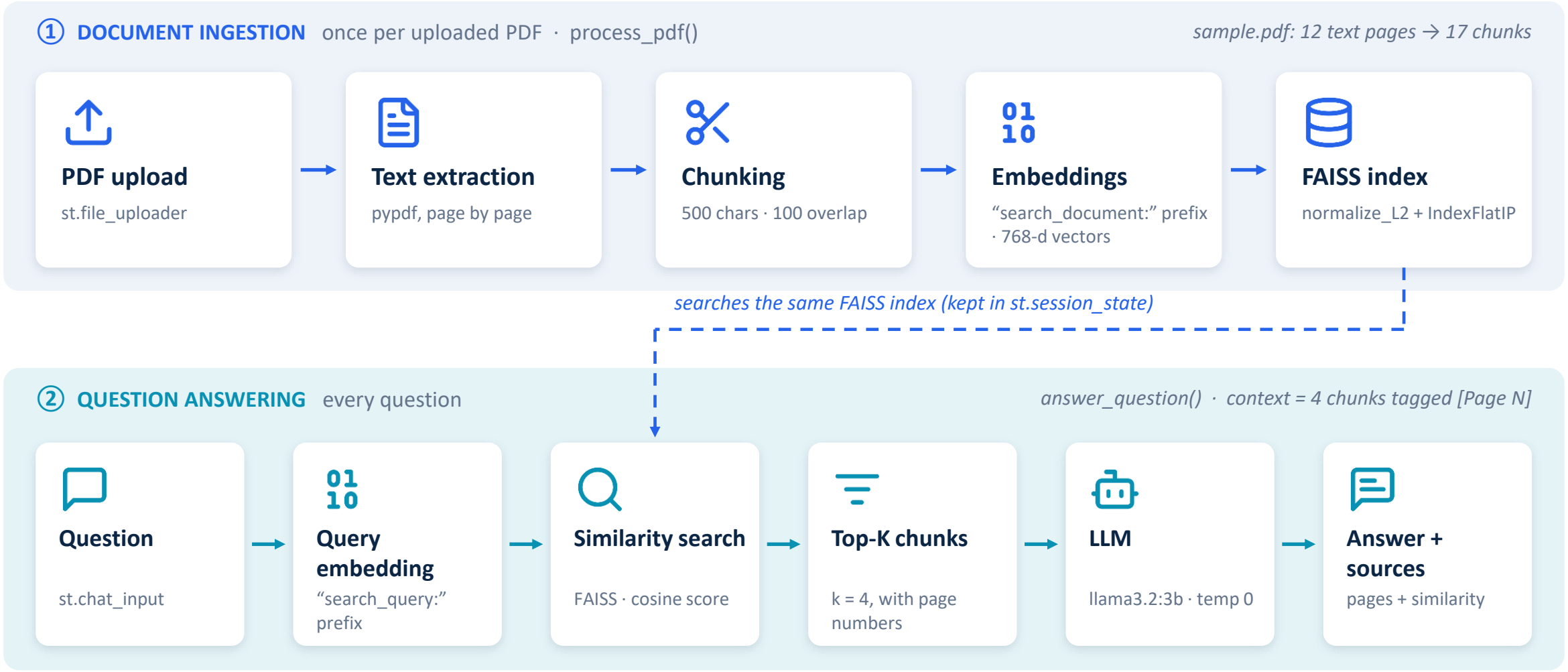
Streamlit

`file_uploader · chat_input`

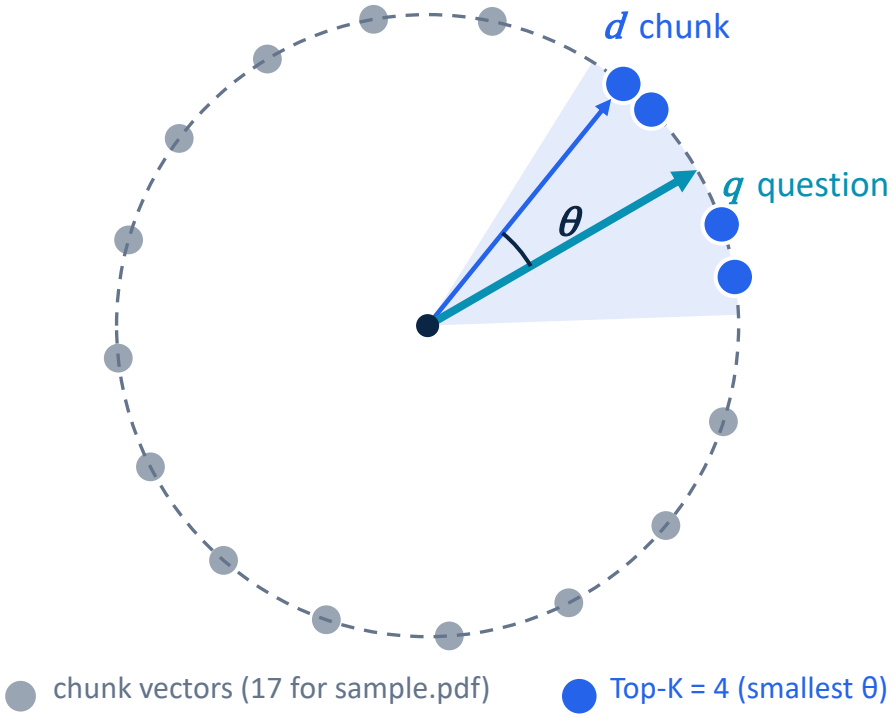
Web UI: upload, chat, source pages and retrieved text

Ollama serves both models locally (localhost:11434): no API key, no internet, documents never leave the laptop.

System Architecture & Working Pipeline



Embeddings, FAISS and the LLM



2-D sketch: real vectors have 768 dimensions

$$\cos\theta = q \cdot d \,/\, (\|q\|\,\|d\|) = q \cdot d$$ after normalize_L2

Embeddings

MAKES VECTORS

nomic-embed-text turns text into a 768-number vector; similar meaning → nearby vectors.

`search_document: <chunk>` `search_query: <question>`

FAISS

SEARCHES VECTORS

Stores the chunk vectors (FAISS id i ↔ chunk i). IndexFlatIP compares the query with every vector (exact search) and returns the Top-K = 4 highest scores.

LLM: llama3.2:3b

WRITES THE ANSWER

Answers only from the 4 retrieved chunks, tagged [Page N]. If the answer isn't there, it must reply with a fixed “not available” sentence.

Key Code from rag_pipeline.py

1 Load & split the PDF

pypdf reads each page → overlapping chunks that keep page metadata

```
reader = PdfReader(pdf_file)
page_texts = [page.extract_text() or "" for page in reader.pages]
splitter = RecursiveCharacterTextSplitter(
    chunk_size=chunk_size,
    chunk_overlap=chunk_overlap,
    separators=["\n\n", "\n", " ", ""],
)
chunks = splitter.split_documents(documents)
```

2 Embed, index & search with FAISS

Prefixed chunks → vectors → IndexFlatIP; the query is searched the same way

```
texts = [DOCUMENT_PREFIX + chunk.page_content for chunk in chunks]
vectors = embedding_model.embed_documents(texts)
faiss.normalize_L2(vectors)                # length 1 -> IP = cosine
self.index = faiss.IndexFlatIP(dimension) # exact "flat" index
self.index.add(vectors)
query_vector = embed_question(question, self.embedding_model)
faiss.normalize_L2(query_vector)
scores, ids = self.index.search(query_vector, k)
```

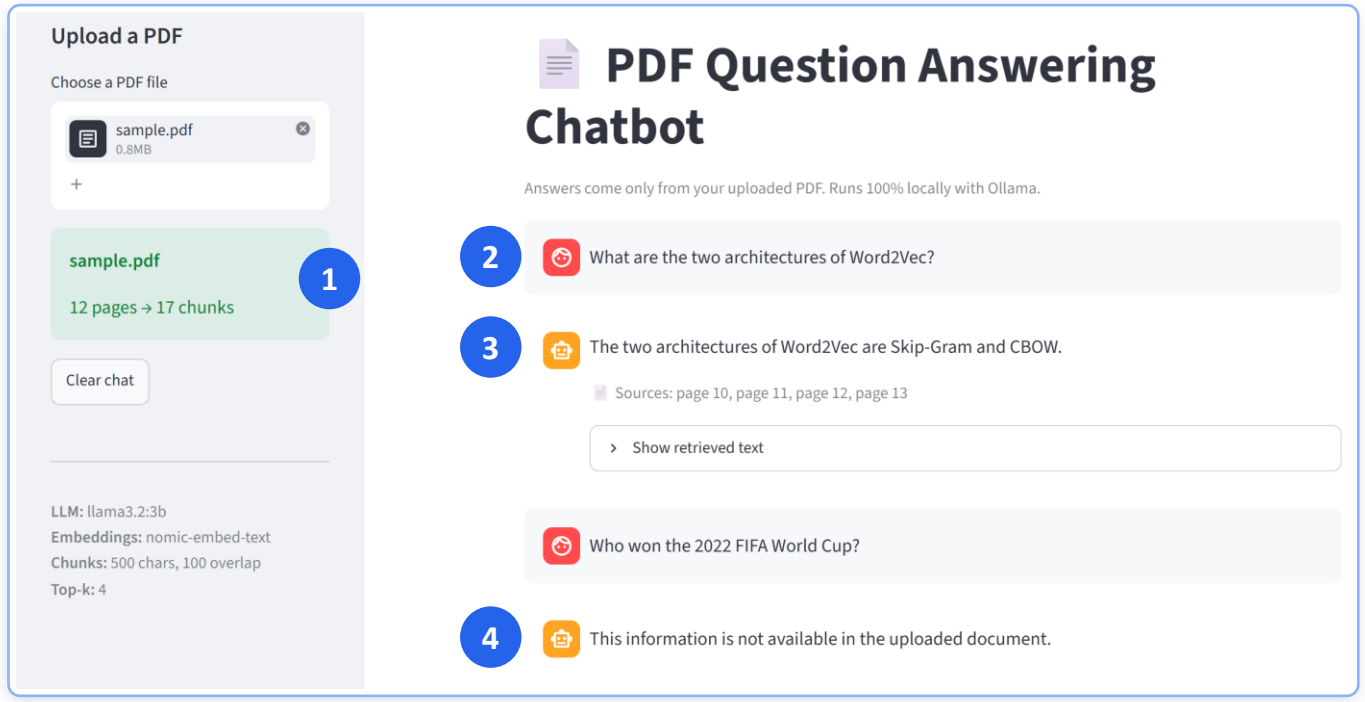
3 Prompt + LLM via a LangChain LCEL chain

Top-4 chunks fill {context}; prompt | llm | parser returns the answer

```
RAG_PROMPT = ChatPromptTemplate.from_messages([
    ("system",
     "You are a question-answering assistant ... "
     "1. Use only facts stated in the CONTEXT. ... "
     "2. If the CONTEXT does not contain the answer, ... "
     "reply with exactly this sentence ...: \"{not_found}\""
     ...),
    ("human", "CONTEXT:\n{context}\n\nQUESTION: {question}"),
])
# get_llm()
return ChatOllama(model=LLM_MODEL, base_url=OLLAMA_BASE_URL,
                  temperature=0, num_ctx=4096)
# answer_question()
chain = RAG_PROMPT | llm | StrOutputParser()
answer = chain.invoke({
    "context": context, "question": question,
    "not_found": NOT_FOUND_MESSAGE,
}).strip()
```

Guardrail: if the reply is the “not available” sentence, the app hides the sources.

Application Demo, Results & Limitations



1. Upload PDF → 12 pages, 17 chunks
2. Question typed in chat
3. Answer + source pages 10–13
4. Out-of-scope question → refused

Test run on sample.pdf

13-page NLP lecture PDF · rag_pipeline.py self-test

17 chunks indexed from 12 text pages

3 / 3 in-document questions answered from the PDF, with pages

2 / 2 out-of-scope questions refused (“not available”)

Top-4 cosine scores for the Word2Vec question:
0.72 · 0.71 · 0.70 · 0.69 (pages 12, 13, 11, 10)

Limitations

- Scanned or image-only PDFs need OCR
- Fixed 500-char chunks can split ideas
- A small 3B model can still hallucinate
- Local inference adds latency; one PDF at a time

Future Scope & Conclusion

More formats + OCR

DOCX, PPTX and scanned PDFs

Many PDFs, saved index

Index all college PDFs once; save FAISS to disk

Better retrieval

Hybrid keyword + vector search, then reranking

Conversation memory

Follow-up questions that use the chat history

Finer citations

Sentence-level citations (pages already shown)

Evaluate & deploy

Labelled test set with metrics; campus server

CONCLUSION

A fully local RAG assistant that answers only from your PDF, shows its sources, and says so when the answer isn't there.

Thank You

Questions & Answers